NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

SEXPERIMENT NO :- 10

AIM :- To design and implement a Gated Recurrent Unit (GRU) model for sequence learning and prediction applications.

OBJECTIVE :-

1. To understand the concept of Gated Recurrent Units.
2. To understand sequence learning using GRU networks.
3. To understand the role of update and reset gates in GRU.
4. To design a GRU model for sequence prediction.
5. To train the GRU model using sequential data.
6. To visualize the actual and predicted values.

THEORY :-

1. Gated Recurrent Unit (GRU)

A Gated Recurrent Unit (GRU) is a type of Recurrent Neural Network (RNN) designed to process sequential data. GRU networks can learn dependencies between different time steps while using a simpler architecture than LSTM. GRUs use gating mechanisms to control the flow of information and can learn long-term dependencies with fewer parameters. They are commonly used in sequence learning, time series forecasting, natural language processing and speech-related applications.

GRU is considered a simplified alternative to LSTM because it does not use a separate cell state. Instead, it maintains a hidden state that carries information through the sequence. This simpler structure can make GRUs faster to train and more computationally efficient than LSTMs.

2. Sequential Data

Sequential data is data in which the order of observations is important. Examples include temperature readings, stock prices, sales data, speech signals and text. A sequence can be represented as:

$$X = [x1, x2, x3, \ldots, xt]$$

where each value represents an observation at a particular time step. In prediction applications, previous values can be used to predict a future value. For example:

$$[x1, x2, x3, x4] \rightarrow x5$$

The GRU learns this relationship by processing the sequence in its original order.

3. GRU Architecture

The GRU architecture mainly consists of two gates: Update Gate and Reset Gate. These gates determine how much previous information should be retained and how much new information should be used. This allows the GRU to selectively remember useful information while ignoring irrelevant information.

Unlike an LSTM, which uses input, forget and output gates along with a separate cell state, the GRU combines the required functions into two gates and maintains a single hidden state. This reduces the number of parameters and makes the architecture comparatively simpler.

4. Update Gate

The update gate determines how much information from the previous hidden state should be retained for the current time step. It can be represented as:

$$zt = \sigma(Wz[ht-1, xt] + bz)$$

where xt is the current input and ht−1 is the previous hidden state. The sigmoid function produces values between 0 and 1. A value closer to 1 indicates that more previous information should be retained, while a value closer to 0 allows more new information to be used.

5. Reset Gate

The reset gate determines how much of the previous hidden state should be ignored when calculating new information. It can be represented as:

$$rt = \sigma(Wr[ht-1, xt] + br)$$

The reset gate is useful when the previous information is no longer relevant to the current input. It allows the GRU to forget certain previous information and focus on the current sequence.

6. Candidate Hidden State

The GRU uses the reset gate to calculate a candidate hidden state containing new information. A simplified representation is:

$$\tilde{ht} = tanh(Wh[rt * ht-1, xt] + bh)$$

The candidate hidden state represents the information that could be stored in the current hidden state.

7. Hidden State

The final hidden state is obtained by combining the previous hidden state and the candidate hidden state according to the update gate.

$$ht = (1 - zt) * ht-1 + zt * \tilde{ht}$$

The hidden state carries information from the current time step to the next time step. Therefore, it acts as the memory of the GRU network.

8. GRU for Sequence Prediction

GRUs can be used for predicting future values from previous observations. A sequence of past values is divided into input sequences and corresponding target values. For example, if the sequence length is 10:

$$[x1, x2, ..., x10] \rightarrow x11$$

The next sequence can be:

$$[x2, x3, ..., x11] \rightarrow x12$$

This sliding-window method creates multiple training examples from a single sequence. The GRU learns the relationship between previous observations and the next value. GRUs are commonly applied to time series forecasting and other sequential prediction tasks.

9. Data Preprocessing

Before training the GRU, the sequential data can be normalized to a suitable numerical range. Normalization helps the model train more efficiently. The Min-Max normalization can be represented as:

$$x' = (x - x_min) / (x_max - x_min)$$

The normalized sequence is then divided into input sequences and target values.

10. Training the GRU Model

The model can be compiled using the Adam optimizer and Mean Squared Error (MSE) loss function for numerical prediction.

$$MSE = (1/n) \Sigma (yi - \hat{y}i)^2$$

where yi is the actual value and $\hat{y}$i is the predicted value. During training, the GRU processes the input sequences and adjusts its weights using backpropagation through time. The objective is to reduce the difference between the actual and predicted values.

11. Working of GRU

The working process can be summarized as follows: generate or load sequential data; normalize the data; create input sequences using a fixed sequence length; use the previous observations as input and the next observation as the target; provide the sequences to the GRU layer; the reset gate determines which previous information should be ignored; the update gate determines which information should be retained; the GRU generates a new hidden state; the Dense layer produces the prediction; calculate the prediction error; update the model weights during training; compare the actual and predicted values using a graph.

PROCEDURE :-

1. Import TensorFlow, NumPy and Matplotlib.
2. Generate the sequential time series data.
3. Normalize the data.
4. Create input sequences and target values.
5. Divide the sequences into training and testing data.
6. Create the GRU model.
7. Add a GRU layer with 50 units.
8. Add a Dense layer for prediction.
9. Compile the model using Adam optimizer and MSE loss.
10. Train the GRU model using the training sequences.
11. Predict the values for the test sequences.
12. Visualize the actual and predicted values.
13. Plot the training and validation loss.

CODE :-

```
import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt


x = np.linspace(0, 100, 1000)

data = np.sin(x)


data_min = data.min()

data_max = data.max()


scaled_data = (data - data_min) / (data_max - data_min)


sequence_length = 20


X = []

y = []


for i in range(len(scaled_data) - sequence_length):

    X.append(scaled_data[i:i + sequence_length])

    y.append(scaled_data[i + sequence_length])


X = np.array(X)

y = np.array(y)
```

```
X = X.reshape((X.shape[0], X.shape[1], 1))

split = int(len(X) * 0.8)

X_train = X[:split]
y_train = y[:split]

X_test = X[split:]
y_test = y[split:]

model = tf.keras.Sequential([
    tf.keras.Input(shape=(sequence_length, 1)),
    tf.keras.layers.GRU(50),
    tf.keras.layers.Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse"
)

history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)

predictions = model.predict(X_test, verbose=0)

predictions = predictions * (data_max - data_min) + data_min
actual = y_test * (data_max - data_min) + data_min

test_loss = np.mean((actual - predictions.flatten()) ** 2)
```

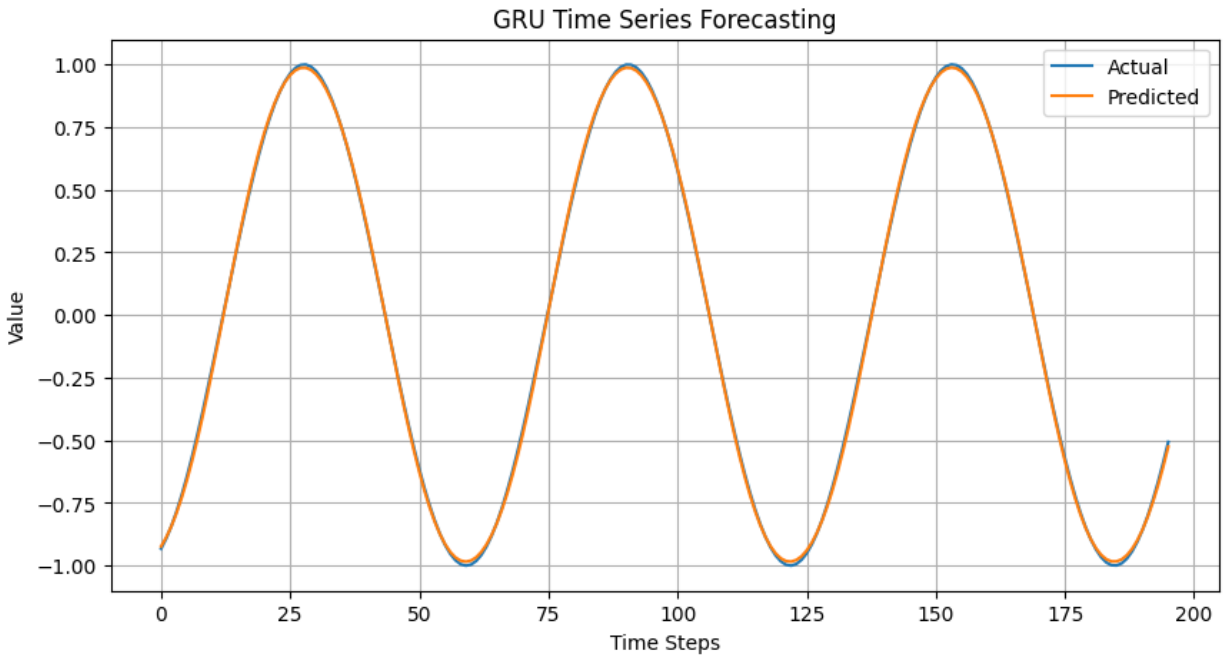
```
print("Test MSE:", test_loss)

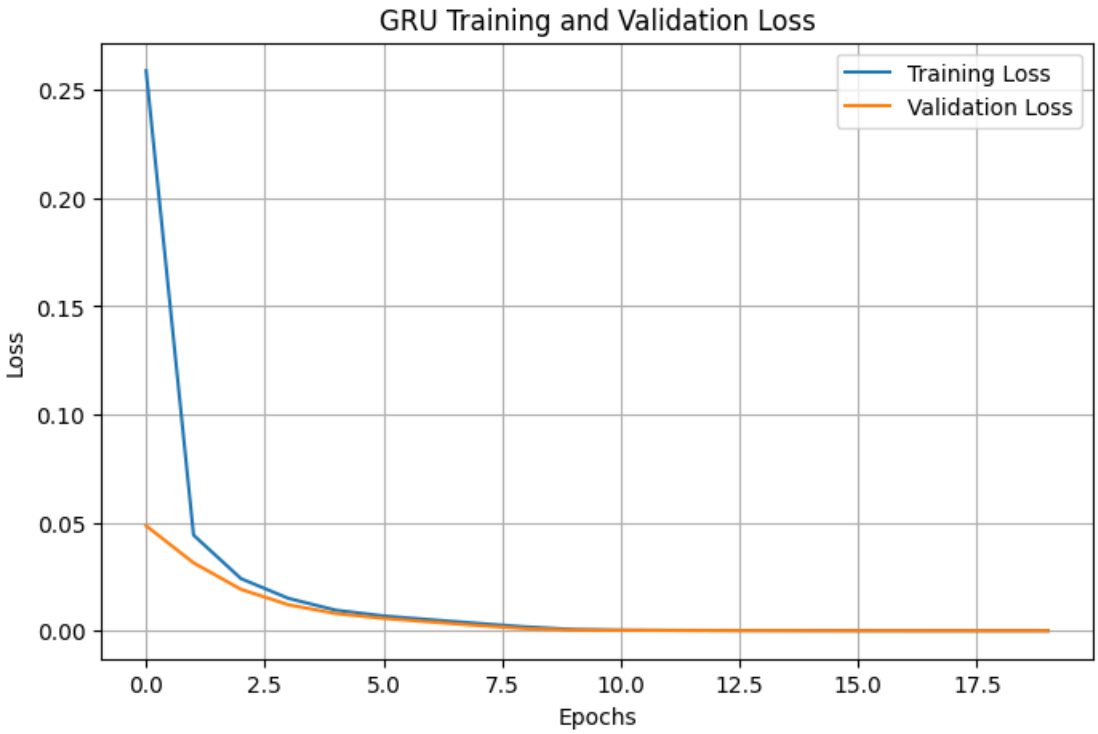

plt.figure(figsize=(10, 5))
plt.plot(actual, label="Actual")
plt.plot(predictions.flatten(), label="Predicted")
plt.xlabel("Time Steps")
plt.ylabel("Value")
plt.title("GRU Time Series Forecasting")
plt.legend()
plt.grid(True)
plt.show()


plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("GRU Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT :-

```
Epoch 1/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - loss: 0.2589 - val_loss: 0.0486
Epoch 2/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0442 - val_loss: 0.0314
Epoch 3/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0241 - val_loss: 0.0192
Epoch 4/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0150 - val_loss: 0.0120
Epoch 5/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0095 - val_loss: 0.0080
Epoch 6/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0069 - val_loss: 0.0058
Epoch 7/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0051 - val_loss: 0.0041
Epoch 8/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0034 - val_loss: 0.0024
Epoch 9/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0018 - val_loss: 0.0010
Epoch 10/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.4221e-04 - val_loss: 3.4694e-04
Epoch 11/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.0990e-04 - val_loss: 2.4707e-04
Epoch 12/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 2.2178e-04 - val_loss: 1.7180e-04
Epoch 13/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 1.5446e-04 - val_loss: 1.1788e-04
Epoch 14/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 1.0647e-04 - val_loss: 7.9614e-05
Epoch 15/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.2925e-05 - val_loss: 5.4952e-05
Epoch 16/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 5.1463e-05 - val_loss: 4.0065e-05
Epoch 17/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 3.8731e-05 - val_loss: 3.1724e-05
Epoch 18/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 3.1544e-05 - val_loss: 2.7197e-05
Epoch 19/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 2.7529e-05 - val_loss: 2.4690e-05
Epoch 20/20
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 2.5177e-05 - val_loss: 2.3159e-05
Test MSE: 9.8819344108233e-05
```




GRU Training and Validation Loss
Training Loss
Validation Loss
Loss
Epochs
0.25
0.20
0.15
0.10
0.05
0.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

CONCLUSION :-

The experiment successfully implemented a GRU model for sequence learning and prediction. The GRU learned temporal patterns from sequential data using update and reset gates and generated predictions for future values. The actual and predicted values were visualized to evaluate the performance of the model.